```
# ====================================================
# AI PRACTICAL 2
# N-Queens Problem using Hill Climbing
# ====================================================


# STEP 1 : NUMBER OF QUEENS

N = 4


# STEP 2 : FUNCTION TO CHECK SAFE POSITION

def is_safe(board, row, col):

    # CHECK LEFT SIDE

    for i in range(col):

        if board[row][i] == 1:

            return False


    # CHECK UPPER DIAGONAL

    i = row
    j = col

    while i >= 0 and j >= 0:

        if board[i][j] == 1:

            return False

        i -= 1
        j -= 1


    # CHECK LOWER DIAGONAL

    i = row
    j = col

    while i < N and j >= 0:

        if board[i][j] == 1:

            return False

        i += 1
        j -= 1


    return True


# STEP 3 : SOLVE FUNCTION
```

```
def solve(board, col):
    if col >= N:
        return True

    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = 1

            if solve(board, col + 1):
                return True

            board[i][col] = 0

    return False

# STEP 4 : CREATE BOARD
board = []
for i in range(N):
    row = []
    for j in range(N):
        row.append(0)
    board.append(row)

# STEP 5 : CALL FUNCTION
solve(board, 0)

# STEP 6 : DISPLAY OUTPUT
print("N Queens Solution\n")
for row in board:
    print(row)
```

```
# ======================================================
# WATER JUG PROBLEM
# ======================================================


# CAPACITY OF JUGS

jug1 = 4
jug2 = 3
target = 2


# INITIAL STATE

x = 0
y = 0


print("\nWater Jug Solution\n")


while x != target and y != target:


    if x == 0:

        x = jug1

        print("Fill Jug1 :", x, y)


    elif y == jug2:

        y = 0

        print("Empty Jug2 :", x, y)


    else:

        transfer = min(x, jug2 - y)

        x = x - transfer

        y = y + transfer

        print("Transfer :", x, y)


print("\nTarget Achieved")
```